

OL Portal — Running Costs at 10 / 25 / 100 / 500 Users

Last updated: July 21, 2026. All prices USD, monthly unless noted. Grounded in what is actually deployed: AWS stack `ol-portal-backend` (us-east-1, account 371361645693), the Claude API (Sonnet 5 for The Optimist, Haiku 4.5 for the file analyzer — switched from all-Opus on July 21, 2026), and the QuickBooks Online integration in `backend/src/qbo.mjs`.

The one-line summary: **the Claude API is ~90% of the cost at every tier.** AWS infrastructure is nearly free until ~100 users, and the Intuit API charges nothing per call.

1. Cost per AI action (the unit economics)

The two AI features run on different models, matched to what each needs:

Feature	Model	Input / Output per MTok
The Optimist (proposal writing)	claude-sonnet-5	\$3 / \$15 — intro pricing \$2 / \$10 through Aug 31, 2026
File analyzer (classify + summarize)	claude-haiku-4-5	\$1 / \$5

The Optimist uses adaptive thinking; thinking tokens bill as output tokens. Numbers below use Sonnet 5's standard (\$3/\$15) pricing — knock ~33% off the Optimist lines through August 2026 while intro pricing lasts.

The Optimist (`POST /assist`, `assist.mjs`)

Every chat message sends: system prompt (~800 tokens) + the full knowledge base (each KB entry up to 30k chars) + deal context + the current draft of all 6 sections + up to the last 30 conversation turns. `max_tokens`: 6000.

Per Optimist message (Sonnet 5, standard pricing)	Tokens (typical)	Cost
Input (system + KB + draft + history)	6,000–15,000	\$0.018–\$0.045
Output (reply + section JSON + thinking)	1,500–4,000	\$0.023–\$0.06
Total per message		~\$0.04–\$0.10 (avg ~\$0.06; ~\$0.04 on intro pricing)

- Input grows as the conversation and draft grow — later messages in a long proposal chat cost more than early ones.
- **A full proposal** built through ~15 Optimist messages: **~\$0.90** (range \$0.60–\$1.80 depending on chat length and draft size).
- **Attachments** add one-time input cost on that message: a PDF bills roughly 1,500–3,000 tokens/page, so a 20-page attached document adds ~\$0.12–\$0.18. Text files are capped at 150k chars (~\$0.12 max). "Auto-fill the rest" is a normal message with a large output (~full 6 sections) — budget ~\$0.10.

File analyzer (`ol-portal-analyzer`, **S3-triggered**)

One Claude call per uploaded file ≤10MB (larger files are stored without analysis). `max_tokens`: 8192, runs on Haiku 4.5.

Per file analyzed (Haiku 4.5)	Tokens (typical)	Cost
Input (document content)	5,000–40,000 (5–25 page doc)	\$0.005–\$0.04
Output (docType/summary/keyPoints)	800–2,000	\$0.004–\$0.01
Total per file		~\$0.01–\$0.05 (avg ~\$0.03); dense 100+ page PDFs up to ~\$0.20

2. Usage assumptions per tier

Internal users split across Admin / Lab Leader / Contributor. Contributors (bench-only) generate almost no AI usage; Admins and Lab Leaders drive it. Assumed monthly averages **per user** (blended across roles):

Assumption	Value
Optimist messages	30 per user (≈ 2 proposals × 15 messages)
File uploads analyzed	5 per user
API requests (page loads, saves, bootstrap)	~2,000 per user
Blended AI cost per user	~ \$1.95/mo (heavy Lab Leaders can be \$5–8; Contributors near \$0)

If real usage skews heavier (more proposals per Lab Leader, big attachments), scale the Claude line linearly — it is almost exactly \$0.06 per Optimist message and \$0.03 per file.

3. Monthly cost by tier

Claude API (billed by Anthropic, not AWS — key lives in SSM `/ol-portal/anthropic-api-key`)

	10 users	25 users	100 users	500 users
Optimist on Sonnet 5 (30 msgs/user × ~\$0.06)	\$18	\$45	\$180	\$900
File analysis on Haiku (5 files/user × ~\$0.03)	\$1.50	\$4	\$15	\$75
Claude total	~\$20	~\$49	~\$195	~\$975
While Sonnet 5 intro pricing lasts (through Aug 2026)	~\$14	~\$34	~\$135	~\$675
Range (light ↔ heavy usage)	\$10–\$40	\$25–\$100	\$100–\$400	\$500–\$2,000

AWS infrastructure (account 371361645693)

Service	10 users	25 users	100 users	500 users	Notes
Lambda (ol-portal-api, 2048MB, holds 10–30s per assist call)	\$0.20	\$0.50	\$2	\$8	The assist route dominates: ~40 GB-s × \$0.0000133 ≈ \$0.0005/call. Everything else is <150ms. Mostly inside the 400k GB-s free tier at small tiers
Lambda (analyzer 512MB, auth-events, recurring cron)	<\$0.10	<\$0.25	\$1	\$4	
API Gateway (HTTP API, \$1/M requests)	\$0.02	\$0.05	\$0.20	\$1	
DynamoDB (ol-portal, on-demand + PITR)	\$0.25	\$0.50	\$2	\$8	Bootstrap reads whole table per session; still tiny
S3 (ol-portal-files-..., 50MB/file cap)	\$0.05	\$0.15	\$1	\$5	~5 files × 5MB/user stored, plus requests
Cognito (Essentials tier)	\$0	\$0	\$0	\$0	First 10,000 MAU free — all tiers fit
Amplify Hosting (builds + data out)	\$0.50	\$0.75	\$2	\$6	Small static frontend; builds \$0.01/min
CloudWatch Logs	\$0.25	\$0.50	\$2	\$8	Assist/analyzer logs grow with AI usage
SSM Parameter Store (standard)	\$0	\$0	\$0	\$0	
AWS total	~\$1.50	~\$3	~\$10	~\$40	

*The **\$10/mo AWS budget alert** on the OL account will start tripping around the 100-user tier. Raise it before scaling past ~75 users so it stays a real anomaly signal (suggested: 2× the expected AWS spend for the current tier).*

Intuit / QuickBooks API

The QBO integration makes these calls (qbo.mjs):

Call	Endpoint	When	Per-call cost
OAuth authorize redirect	appcenter.intuit.com/connect/oauth2	Once, when an admin connects	\$0
Token exchange	oauth.platform.intuit.com/oauth2/v1/tokens/bearer	Once at connect	\$0
Token refresh	same bearer endpoint	Access tokens last 1 hour; refreshed on demand before reads	\$0
Invoice read	GET /v3/company/{realmId}/query — SELECT * FROM Invoice MAXRESULTS 100	Each time an admin opens the QuickBooks card on invoices.html	\$0

Call	Endpoint	When	Per-call cost
Status check	(local token lookup, no Intuit call)	Page load	\$0

The QuickBooks Online Accounting API has no per-call pricing. Costs are:

Item	Cost	Notes
Intuit developer account + sandbox	\$0	Current QBO_ENV=sandbox costs nothing
API calls (production)	\$0	Free with a QBO company subscription; rate limit 500 requests/min per realm — the portal's admin-only reads use a tiny fraction even at 500 users
QuickBooks Online subscription (production prerequisite)	~\$38–\$99/mo depending on plan (verify current Intuit pricing)	Whoever owns the connected books (OL) pays this regardless of the portal; it is not a per-user portal cost

QBO cost is **flat across all four tiers** — only admins hit /qbo/*, and there is one connected realm. The only action item for production is the redirect-URI registration + prod client secret (already documented in the repo); no cost impact.

4. Total monthly cost summary

	10 users	25 users	100 users	500 users
Claude API (Anthropic invoice)	\$20	\$49	\$195	\$975
AWS (OL account)	\$1.50	\$3	\$10	\$40
Intuit API	\$0	\$0	\$0	\$0
Total	~\$21	~\$52	~\$205	~\$1,015
Per user	~\$2.15	~\$2.10	~\$2.05	~\$2.03

Cost scales almost perfectly linearly with users because it is dominated by per-message Claude spend. There is no meaningful fixed-cost floor beyond ~\$1/mo.

5. Model choice — how swapping Claude models changes cost

Every current Claude model keeps the same 1:5 input/output price ratio, so model choice scales each feature's cost by a simple multiplier:

Model	Input / Output per MTok	vs Opus 4.8	Optimist msg	File analyzed
Claude Fable 5 (c laude - fable - 5)	\$10 / \$50	2.0×	~\$0.20	~\$0.30
Claude Opus 4.8 (c laude - opus - 4 - 8)	\$5 / \$25	1.0×	~\$0.10	~\$0.15
Claude Sonnet 5 (c laude - sonnet - 5) — current Optimist model	\$3 / \$15	0.6×	~\$0.06	~\$0.09

Model	Input / Output per MTok	vs Opus 4.8	Optimist msg	File analyzed
Claude Sonnet 5 — intro pricing through Aug 31, 2026	\$2 / \$10	0.4×	~\$0.04	~\$0.06
Claude Haiku 4.5 (claude-haiku-4-5) — current analyzer model	\$1 / \$5	0.2×	~\$0.02	~\$0.03

Monthly Claude spend by scenario

Scenario	10 users	25 users	100 users	500 users
Everything on Opus 4.8 (config before July 21, 2026)	\$38	\$94	\$375	\$1,875
Analyzer → Haiku, Optimist on Opus	\$32	\$79	\$315	\$1,575
Current: Optimist → Sonnet 5, analyzer → Haiku	\$20	\$49	\$195	\$975
Everything → Sonnet 5	\$23	\$56	\$225	\$1,125
Everything → Haiku 4.5	\$8	\$19	\$75	\$375
Everything → Fable 5	\$75	\$188	\$750	\$3,750

Why this configuration

- **Analyzer on Haiku 4.5:** classification + summary + key-point extraction is well within Haiku's range — 80% cheaper than Opus with minimal quality risk. (Haiku's 200K context window is still far more than any ≤10MB doc the analyzer accepts. Haiku doesn't support adaptive thinking, so the analyzer runs without a thinking parameter.)
- **Optimist on Sonnet 5:** near-Opus quality on writing and agentic work at 0.6× the price (0.4× on intro pricing through Aug 2026). If proposal quality noticeably drops on real deals, switching back is a one-line change in assist.mjs (model: "claude-opus-4-8") + sam deploy — the ~\$1,500/mo question only matters at the 500-user tier.
- **Haiku for the Optimist is not recommended** — interview quality and section prose would noticeably weaken. **Fable 5** doubles Opus cost and targets long-horizon agentic work; overkill for both features.
- Caveat: token counts differ slightly across models (Haiku uses an older tokenizer), so treat the multipliers as ±10%. The structural cost — re-sending the KB and draft every message — dominates regardless of model, so the prompt-caching lever below stacks with any model choice.

6. Levers if Claude spend needs to come down

Ordered by impact:

- ~~1. Route each feature to the right-sized model.~~ **Done July 21, 2026** — Optimist on Sonnet 5, analyzer on Haiku 4.5 (cut the Claude line ~48% vs all-Opus).
- 2. Prompt caching (up to ~90% off input on repeat calls).** The assist system prompt embeds the live draft, so today no prefix is stable enough to cache. Restructuring so KB + static instructions come first with a cache_control breakpoint (draft + deal context after it) would cut the input side of every Optimist message

substantially — cached reads bill at ~10% of the input rate.

3. Trim what each assist call re-sends. Only include KB entries relevant to the deal's lab, and drop draft sections that haven't changed, instead of shipping everything every message.

4. Per-user rate limiting / monthly caps on /assist before opening the portal to many Lab Leaders, so a runaway chat loop can't create a surprise bill.

5. Set an Anthropic console spend limit mirroring the AWS budget alert — today the Claude spend has no equivalent guardrail.